# Table of Contents

# Table of Contents

1.0 Purpose with the Problem scenario ..... 3

1.1 Introduction ..... 3

1.2 Relating real world scenario using technical Concepts ..... 4

1.3 Detailed Explanation ..... 5

1.3.1 Introduction to ByteStreams ..... 5

1.3.1.1 InputStream: ..... 6

1.3.1.2 OutputStream: ..... 10

..... 14

1.3.2 Introduction to CharacterStreams ..... 14

1.3.2.1 Reader: ..... 17

1.3.2.2 Writer: ..... 22

1.3.3 Scenario Revisited: ..... 27

1.3.4 Using the File Class: ..... 27

1.3.4.1 Changing the Read/Write Permissions of the File ..... 28

1.3.4.2 List out all the files and folders present in a directory ..... 29

1.3.4.3 Print the path (absolute and relative) of a file, and size of the file ..... 31

1.3.4.4 Checking if the given path name represents file or a directory ..... 32

1.3.4.5 Create a new folder and a new file inside the folder, only if the specified file or folder does not exist. ..... 32

..... 32

1.3.4.6 Delete a Specified File. ..... 34

1.3.4.7 Rename a file to new file name. ..... 34

1.3.4.8 Code snippet on File class ..... 35

1.3.5 Scenario Revisited: ..... 39

..... 39

1.3.6 Using DataInputStream,FileOutputStream classes for the given problem scenario: ..... 40

1.3.7 Introduction to Buffer ..... 42

1.3.8 Relating Buffer to Real World Scenario ..... 42

1.3.9 Using DataInputStream ,BufferedOutputStream,FileOutputStream classes for the given problem scenario ..... 44

1.3.10 Code Snippet using DataInputStream ,BufferedOutputStream,FileOutputStream classes ..... 44

1.3.11 Using BufferedInputStream ,BufferedOutputStream,FileOutputStream classes for the given problem scenario ..... 46

1.3.12 Code Snippet using BufferedInputStream ,BufferedOutputStream,FileOutputStream classes 48

1.3.13 Introduction to BufferedReader and BufferedWriter ..... 48

1.3.14 Performing File I/O using BufferedReader and BufferedWriter classes ..... 49

1.3.15 Code snippet using BufferedReader and BufferedWriter classes ..... 50

1.4 Conclusion-- Choosing the right I/O implementation class. ..... 50

1.5 Check your self ..... 52

1.6 Best Practises ..... 53

1.7 Take Away Points ..... 55

1.8 Related Topics ..... 56

1.9 References ..... 56

## 1.0 Purpose with the Problem scenario

Mr .John who works as a developer for ABC Company got a technical requirement in java to develop an payroll based application that accepts employee data from the user using command line and to store the data permanently.

John who never worked on this type of requirement is now looking into finding the possible solutions for the problem

what are the possible alternatives for john?

1)variables?
2)Collection framework classes?

John knows that none of the approaches will work as both the approaches store the data temporarily. But he needs to store the data permanently.

Then after exploring on java packages he found a package **java.io** which is used for performing input,output related operations.

## 1.1 Introduction

java.io package contains various Stream classes that are specifically designed to support input output operations,processing of all types of data.

Input operation means flow of data into the program and output means flow of data out of the program.

**What are the possible ways where a program can get input ? Choose the right option**

1)Keyboard
2)mouse
3)program
4)network
5)memory
6)Disk
7)all the above

Answer would be option 7 because all these are the possible ways where a program can get input.

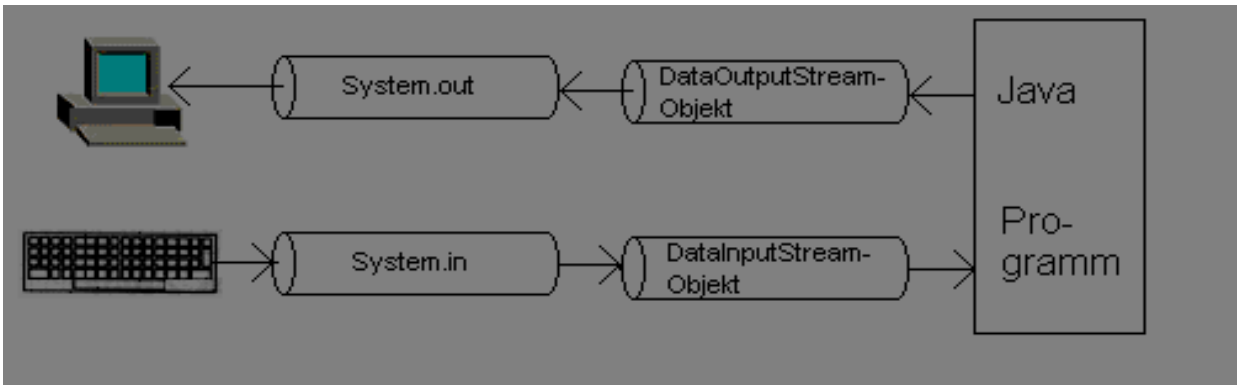
**What are the possible ways where a program can send output? Choose the right option**
1)Screen
2)Printer
3)Memory
4)Disk
5)Network
6)Another program
7)All the above

Answer would be option 7 because all these are the possible ways where a program can send output.

**How the data is flowing to and fro from a program?**

Data cannot flow on its own as it requires a path along which the data can flow. So to provide an uniform,easy to use ,object oriented interface between program and input /output devices. Streams are provided in java.io package

A stream in java is a path along which the data flows




Three streams are created for us automatically and that are available in java.lang.System:

1) System.out: standard output stream
2) System.in: standard input stream
3) System.err: standard error

## 1.2 Relating real world scenario using technical Concepts

We can relate the Stream concept with a river along which water flows.

## 1.3 Detailed Explanation

There are two possible ways where data can be flowed inside the streams

1)Byte Streams(If the data is flown in the form of bytes)

2)Character Streams(If the data is flown in the form of characters)

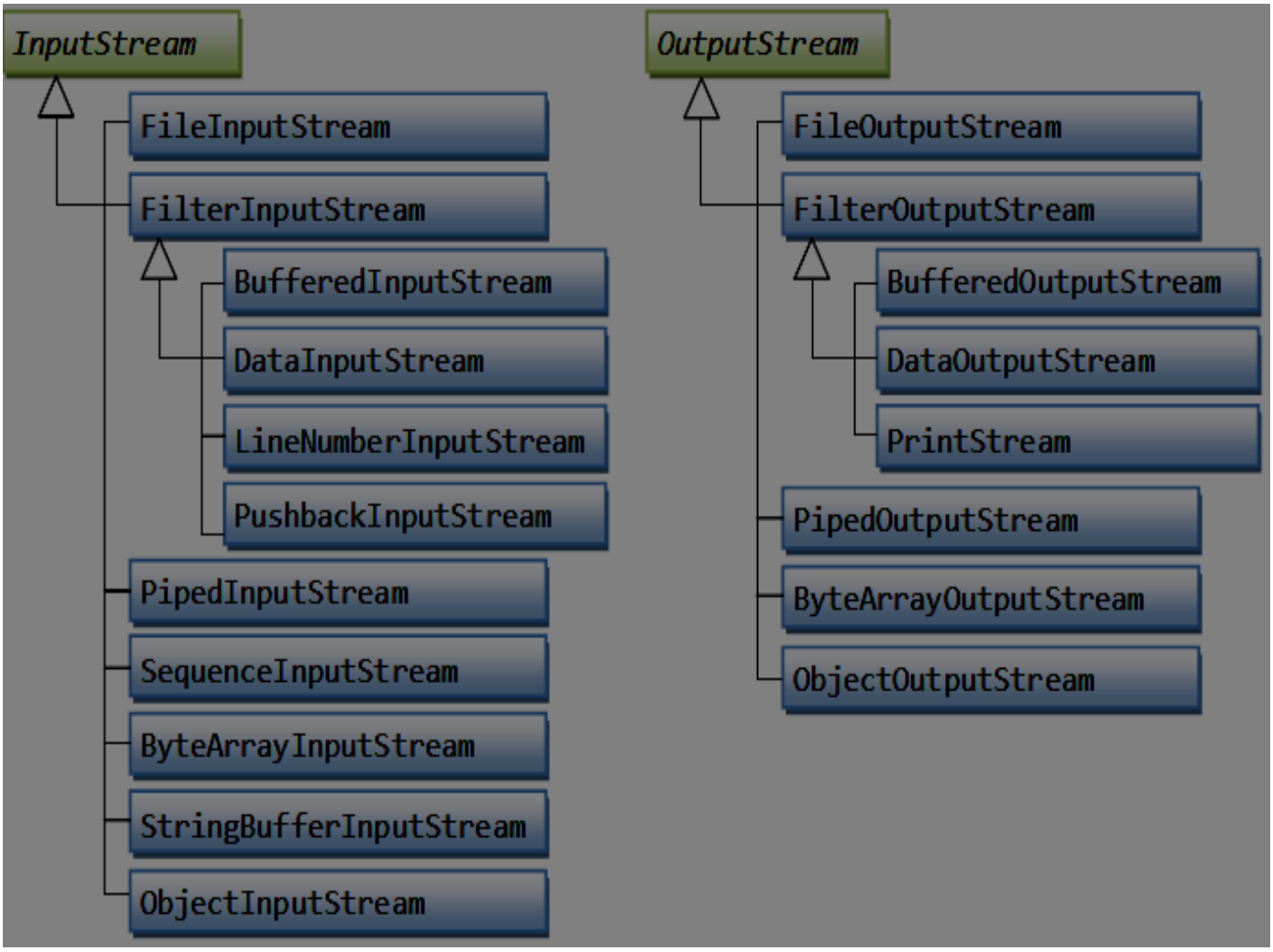
### 1.3.1 Introduction to ByteStreams

ByteStream classes are classified into two:

1)InputStream classes(These classes are used to read the data in the form of 8bit bytes)

2)OutputStream classes (These classes are used to write the data in the form of 8bit bytes)

**Class diagram of Byte Stream classes:**



### 1.3.1.1 InputStream:

InputStream is an abstract class from which all byte-oriented input streams are derived. Because

InputStream is an abstract class,we cannot create instance for this class

What does InputStream class provide?

1)Provides read() method for reading bytes

2)provides method for closing streams

3)Marking positions in the stream

4)skipping ahead in a stream

5)Finding the number of bytes in the stream

Please refer to the below document for complete list of methods of InputStream class

**SubClasses of InputStream :**

1)FileInputStream :This class is used to read the contents from the file in the form of bytes.

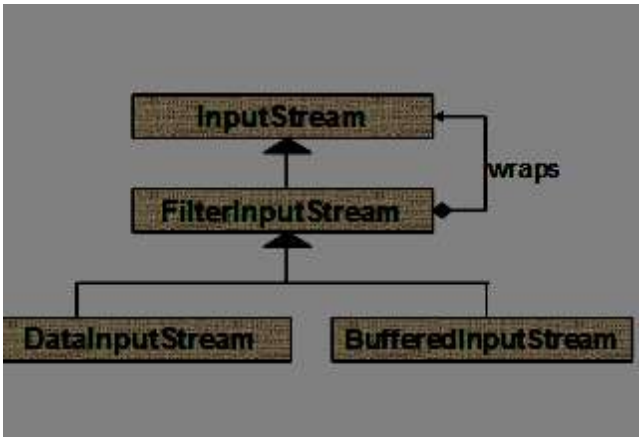
An input stream from a file can be created with the below sample syntax:

```
File file_in = new File ("data.dat");
FileInputStream in = new FileInputStream(file_in);
```

Please refer to the document for more information about FileInputStream class.

2)FilterInputStream: This class provides no additional functionality, but serve as a base class for InputStream  wrappers.

Two commonly used subclasses of FilterInputStream are DataInputStream and BufferedInputStream



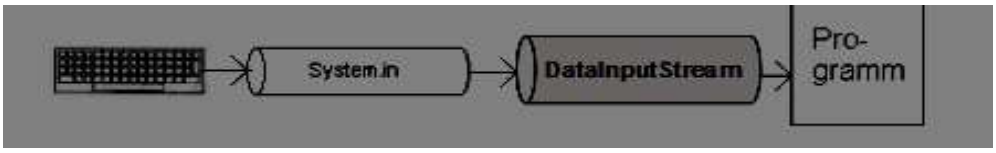

Please refer to the below link for more information about FilterInputStream class.

http://docs.oracle.com/javase/7/docs/api/java/io/FilterInputStream.html

3)DataInputStream: This class extends FilterInputStream, which extends InputStream.

DataInputStream class implements DataInput interface.

This interface reads a sequence of bytes and convert them into values of primitive type




Please refer to the document for more information about DataInputStream class.

4)LineNumberInputStream: This class is an input stream filter that provides the added functionality of keeping track of the current line number.A line is a sequence of bytes ending with a carriage return character ('\r'), a newline character ('\n'), or a carriage return character followed im

mediately by a linefeed character. In all three cases, the line terminating character(s) are re turned as a single newline character.

Please refer to the below link for more information about LineNumberInputStream

http://docs.oracle.com/javase/7/docs/api/java/io/LineNumberInputStream.html

5)BufferedInputStream:

A BufferedInputStream adds functionality to another input stream-namely, the ability to buffer the input and to support the mark and reset methods. When the BufferedInputStream is created, an internal buffer array is created. As bytes from the stream are read or skipped, the internal buf fer is refilled as necessary from the contained input stream, many bytes at a time. The mark op eration remembers a point in the input stream and the reset operation causes all the bytes read since the most recent mark operation to be reread before new bytes are taken from the contained input stream.

Please refer to the document for more information about BufferedInputStream class.

6)PushbackInputStream :A PushbackInputStream adds functionality to another input stream, namely the ability to "push back" or "unread" one byte.

Please refer to the below link for more information about PushbackInputStream

http://docs.oracle.com/javase/7/docs/api/java/io/PushbackInputStream.html

7)PipedInputStream:A piped input stream should be connected to a piped output stream; the piped input stream then provides whatever data bytes are written to the piped output stream.

Please refer to the below link for more information about PipedInputStream

http://docs.oracle.com/javase/7/docs/api/java/io/PipedInputStream.html

8)SequenceInputStream :A SequenceInputStream represents the logical concatenation of other input streams.

Please refer to the below link for more information about SequenceInputStream

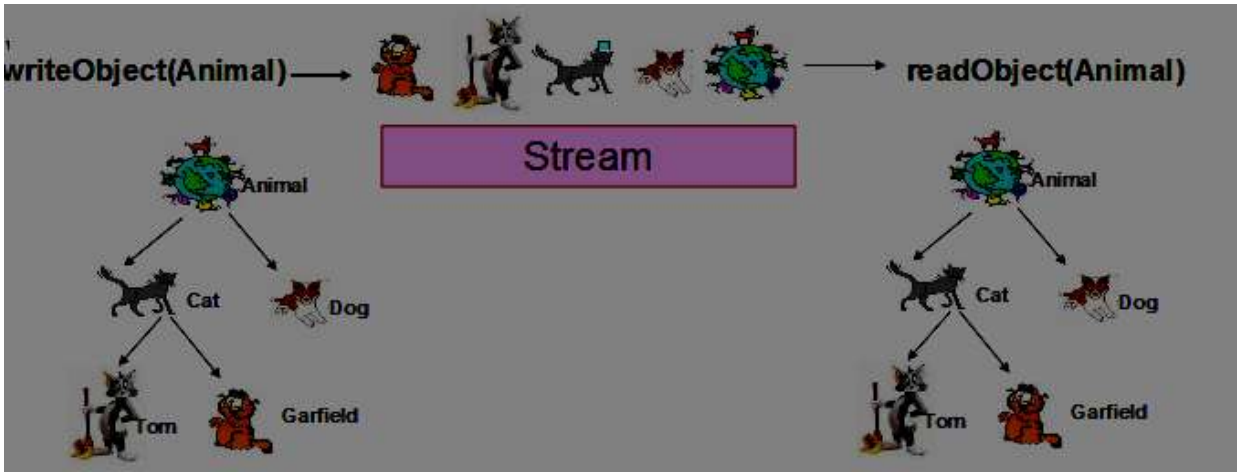
http://docs.oracle.com/javase/7/docs/api/java/io/SequenceInputStream.html

9)ByteArrayInputStream :A ByteArrayInputStream contains an internal buffer that contains bytes that may be read from the stream.

Please refer to the below link for more information about ByteArrayInputStream

http://docs.oracle.com/javase/7/docs/api/java/io/ByteArrayInputStream.html

10)ObjectInputStream: An ObjectInputStream deserializes primitive data and objects previously written using an ObjectOutputStream.




Please refer to the below link for more information about ObjectInputStream

http://docs.oracle.com/javase/7/docs/api/java/io/ObjectInputStream.html

### 1.3.1.2 OutputStream:

OutputStream is an abstract class from which all out **byte**-oriented output streams are derived .

Because OutputStream is an abstract class,we cannot create instance for this class.

What does OutputStream class provide?

1)Writing bytes

2)Closing Streams

3)Flushing Streams

Please refer to the below document for complete list of methods of OutputStream class.

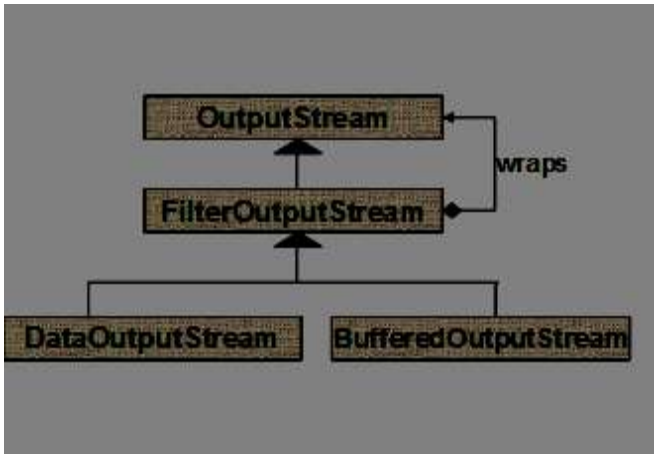
**SubClasses of OutputStream :**

1)FileOutputStream :A file output stream is an output stream for writing data to a File

Please refer to the document for more information about FileOutputStream class.

2)FilterOutputStream: FilterOutputStream provide no additional functionality, but serve as a base class for OutputStream wrappers.

Two commonly used subclasses of FilterOutputStream are BufferedOutputStream and DataOutputStream




Please refer to the below link for more information about FilterOutputStream

http://docs.oracle.com/javase/7/docs/api/java/io/FilterOutputStream.html

3)DataOutputStream:A data output stream lets an application write primitive Java data types to an output stream

Please refer to the below document  for more information about DataOutputStream

4)BufferedOutputStream:The BufferedOutputStream class provides buffering to your output streams. Buffering can speed up IO quite a bit. Rather than write one byte at a time to the net work or disk, you write a larger block at a time. This is typically much faster, especially for disk access and larger data amounts

For Example the below code sets  the internal buffer to 8 KB:

```
OutputStream output = new BufferedOutputStream( new FileOutputStream("c:/data/output-file.txt"), 8 * 1024);
```

Please refer to the document for more information about BufferedOutputStream class

5)PrintStream:The PrintStream class enables you to write formatted data to an underlying Out putStream. For instance, writing int, long and other primtive data formatted as text, rather than as their byte values.

Example:

```
PrintStream output = new PrintStream(new FileOutputStream("c:\text"));
output.print(true);
output.print((int) 123);
output.print((float) 123.456);
output.printf(Locale.UK, "Text + data: %1$", 123);
output.close();
```

Please refer to the below link for more information about PrintStream class:

http://docs.oracle.com/javase/7/docs/api/java/io/PrintStream.html

6)PipedOutputStream:A piped output stream can be connected to a piped input stream to create a communications pipe.

Please refer to the below link for more information about PipedOutputStream:

http://docs.oracle.com/javase/7/docs/api/java/io/PipedOutputStream.html

7)ByteArrayOutputStream: The ByteArrayOutputStream class of the Java IO API allows you to

capture data written to a stream in an array.

For Example:

```
ByteArrayOutputStream output = new ByteArrayOutputStream();

//write data to output stream

byte[] bytes = output.toByteArray();
```

Please refer to the below link for more information about ByteArrayOutputStream:

http://docs.oracle.com/javase/7/docs/api/java/io/ByteArrayOutputStream.html

8)ObjectOutputStream: The ObjectOutputStream class enables you to write Java objects to Out putStream's instead of only bytes. You wrap an OutputStream in a ObjectOutputStream and then you can write objects to it. Here is an example:

```
ObjectOutputStream output = new ObjectOutputStream( new FileOutputStream("object.data"));

MyClass object = new MyClass();

output.writeObject(object);

output.close();
```

This serialized object can now be read via an ObjectInputStream.

Please refer to the below link for more information about ObjectOutputStream:

http://docs.oracle.com/javase/7/docs/api/java/io/ObjectOutputStream.html

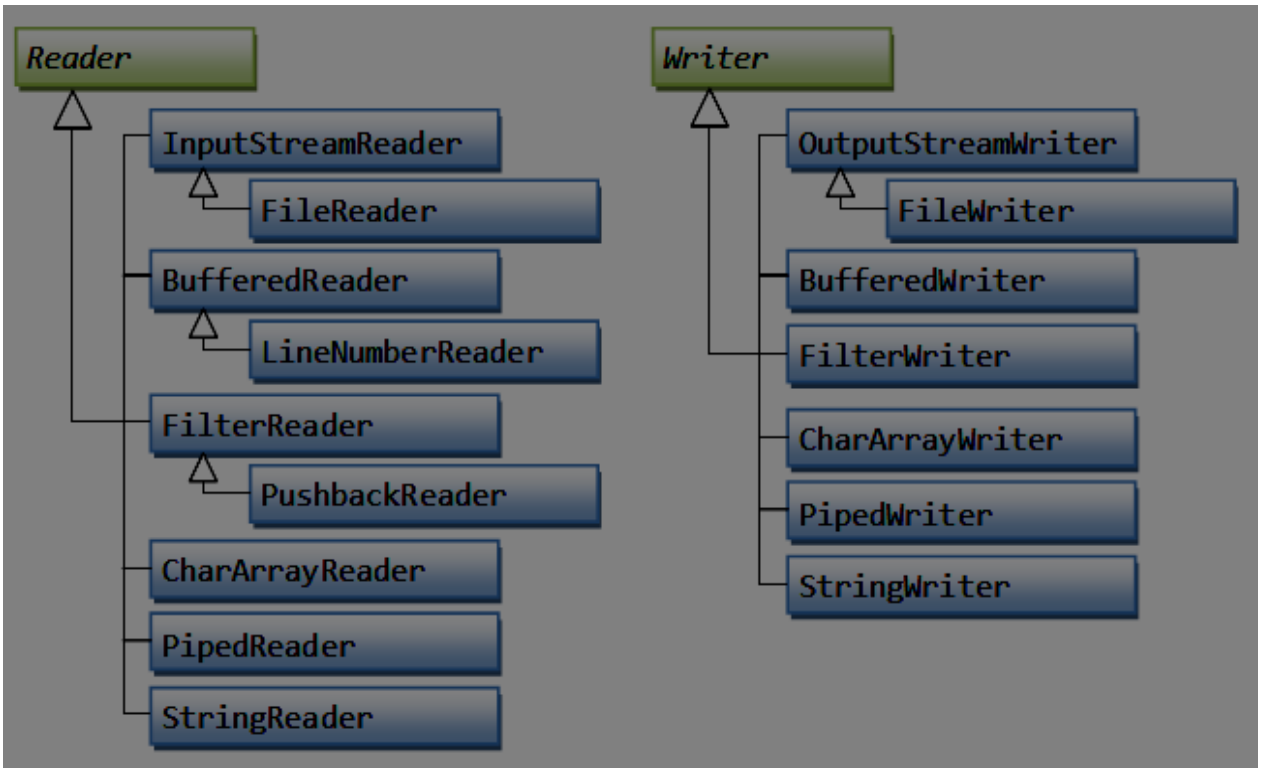
### 1.3.2 Introduction to CharacterStreams

CharacterStream classes are classified into two:

1)Reader classes(These classes are used to read the data in the form of 16 bit unicode charac ters)

2)Writer classes (These classes are used to write the data in the form of 16bit unicode charac ters)

**Class diagram of Character Stream classes:**




#### 1.3.2.1 Reader:

Reader is an abstract class designed to read the characters. Reader contains similar methods as that of InputStream class except it is designed to read characters.

Please refer to the document for more information about Reader

**SubClasses of Reader :**

1)InputStreamReader :The InputStreamReader class is intended to wrap an InputStream, thereby turning the byte based input stream into a character based Reader.

For Example:

```
InputStream inputStream = new FileInputStream("c:\\data\\input.txt");
Reader      reader      = new InputStreamReader(inputStream);

int data = reader.read();
while(data != -1){
    char theChar = (char) data;
    data = reader.read();
}

reader.close();
```

Please refer to the below link for more information about InputStreamReader:

http://docs.oracle.com/javase/7/docs/api/java/io/InputStreamReader.html

2)FileReader: The FileReader class makes it possible to read the contents of a file as a stream of characters. It works much like the FileInputStream except the FileInputStream reads bytes, whereas the FileReader reads characters.

```
Reader reader = new FileReader("c:\\data\\input-text.txt");

int data = reader.read();
while(data != -1) {
  //do something with data...
  doSomethingWithData(data);

  data = reader.read();
}
reader.close();
```

Please refer to the below link for more information about FileReader

http://docs.oracle.com/javase/7/docs/api/java/io/FileReader.html

3)BufferedReader:The BufferedReader class provides buffering to your Reader's. Buffering can speed up IO quite a bit. Rather than read one character at a time from the network or disk, you read a larger block at a time. This is typically much faster, especially for disk access and larger data amounts.The main difference between BufferedReader and BufferedInputStream is that Reader's work on characters (text), wheres InputStream's works on raw bytes.

To add buffering to your Reader's simply wrap them in a BufferedReader. Here is how that looks:

Simply said BufferedReader reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines.

```
Reader input = new BufferedReader(new FileReader("c:\\data\\input-file.txt"));
```

Please refer to the below link for more information about BufferedReader

4)LineNumberReader:The LineNumberReader class is a BufferedReader that keeps track of line

numbers of the read characters. Line numbering begins at 0. Whenever the LineNumberReader encounters a line terminator in the characters returned by the wrapped Reader, the line number is incremented.

You can get the current line number by calling the getLineNumber() method. You can also set the current line number, should you need to, by calling the setLineNumber() method.

For Example:

```
LineNumberReader reader = new LineNumberReader(new FileReader("c:\\data\\input.txt"));
int data = reader.read();
while(data != -1){
    char dataChar = (char) data;
    data = reader.read();
    int lineNumber = reader.getLineNumber();
}
```

Please refer to the below link for more information about LineNumberReader class.

http://docs.oracle.com/javase/7/docs/api/java/io/LineNumberReader.html

5)FilterReader: Abstract class for reading filtered character streams.

Please refer to the below link for more information about FilterReader class.

hhttp://docs.oracle.com/javase/7/docs/api/java/io/FilterReader.html

6)PushbackReader:The PushbackReader is intendended to be used when you parse data from a Reader. Sometimes you need to read ahead a few characters to see what is coming, before you can determine how to interprete the current character. The PushbackReader actually allows you to push back the read characters into the Reader. These characters will then be read again the next time you call read().

The PushbackReader works much like the PushbackInputStream except that the PushbackRead er works on characters, whereas the PushbackInputStream works on bytes.

For Example:

```
PushbackReader reader = new PushbackReader(new FileReader("c:\\data\\input.txt"));

int data = reader.read();

reader.unread(data);
```

Please refer to the below link for more information about PushbackReader class

http://docs.oracle.com/javase/7/docs/api/java/io/PushbackReader.html

7)CharArrayReader:The CharArrayReader class enables you to read the contents of a char array as a character stream

For Example:

```
String s=new String("The CharArrayReader class enables you to read the contents of a char array ");
char[] chars =s.toCharArray();

Reader reader = new CharArrayReader(chars);

int data = reader.read();
while(data != -1) {
  //do something with data

  data = reader.read();
}

reader.close();
```

Please refer to the below link for more information about CharArrayReader class

http://docs.oracle.com/javase/7/docs/api/java/io/CharArrayReader.html

8)PipedReader:The PipedReader class makes it possible to read the contents of a pipe as a stream of characters. As such it works very much like a PipedInputStream except the PipedIn putStream is byte based, not character based.

For Example:

```
Reader reader = new PipedReader(pipedWriter);

int data = reader.read();
while(data != -1) {
  //do something with data...
  doSomethingWithData(data);

  data = reader.read();
}
reader.close();
```

Please refer to the below link  for more information about  PipedReader class.

http://docs.oracle.com/javase/7/docs/api/java/io/PipedReader.html

9)StringReader:The StringReader class enables you to turn an ordinary String into a reader.

For Example:

```
Reader reader = new StringReader("input string...");

int data = reader.read();
while(data != -1) {
  //do something with data...
  doSomethingWithData(data);

  data = reader.read();
}
reader.close();
```

Please refer to the below link  for more information about  StringReader class.

http://docs.oracle.com/javase/7/docs/api/java/io/StringReader.html

### 1.3.2.2 Writer:

Writer is an abstract class from which all out character Stream classes are derived .

What does Writer class provide?

Writer class provides the same methods that are available in OutputStream class.

1)Writing characters

2)Closing Streams

3)Flushing Streams

Please refer to the document for more information about Writer

**SubClasses of Writer :**

1)OutputStreamWriter:The OutputStreamWriter class is intended to wrap an OutputStream, thereby turning the byte based output stream into a character based Writer.

For Example:

```
OutputStream outputStream = new FileOutputStream("c:\\data\\output.txt");
Writer       writer       = new OutputStreamWriter(outputStream);

writer.write("Hello World");

writer.close();
```

Please refer to the below link  for more information about  OutputStreamWriter class.

http://docs.oracle.com/javase/7/docs/api/java/io/OutputStreamWriter.html

2)FileWriter:This class is used for writing characters in to the files.

Please refer to the document for more information about  FileWriter.

http://docs.oracle.com/javase/7/docs/api/java/io/FileWriter.html

3)BufferedWriter:The BufferedWriter class provides buffering to your Writer's. Buffering can speed up IO quite a bit. Rather than write one character at a time to the network or disk, you write a larger block at a time. This is typically much faster, especially for disk access and larger data amounts.

To add buffering to your Writer's simply wrap them in a BufferedWriter.

Example:

```
Writer writer = new BufferedWriter( new FileWriter("c:\\data\\output-file.txt"));
```

Please refer to the link for more information about  BufferedWriter:

http://docs.oracle.com/javase/7/docs/api/java/io/BufferedWriter.html

4)FilterWriter:This class is used for writing filtered  character streams.

Please refer to the link for more information about  FilterWriter:

http://docs.oracle.com/javase/7/docs/api/java/io/FilterWriter.html

5)CharArrayWriter: The CharArrayWriter makes it possible to write characters to a Writer and convert the written characters into a char array.

A CharArrayWriter can be handy in situations where you have a component that outputs charac -ters to a Writer, but you need to access that data as a char array.

Example:

```
CharArrayWriter writer = new CharArrayWriter();

//write characters to writer.

char[] chars = writer.toCharArray();
```

Please refer to the link for more information about CharArrayWriter:

http://docs.oracle.com/javase/7/docs/api/java/io/CharArrayWriter .html

6)PipedWriter:The PipedWriter class makes it possible to write to a Java pipe as a stream of characters. In that respect the PipedWriter works much like a PipedOutputStream except that a PipedOutputStream is byte based, whereas a PipedWriter is character based. The PipedWriter is intended for writing text, in other words.

Example:

```
PipedWriter writer = new PipedWriter(pipedReader);

while(moreData()) {
  int data = getMoreData();
  writer.write(data);
}
writer.close();
```

Please refer to the link for more information about PipedWriter:

http://docs.oracle.com/javase/7/docs/api/java/io/ PipedWriter.html

7)StringWriter:The StringWriter class enables you to obtain the characters written to a Writer as a String.

The method **toString()** of StringWriter class returns the characters written to the String Writer as a String.

The method **getBuffer()** of StringWriter class returns the StringBuffer used by the StringWriter to build the string from the written characters.

Example:

```
StringWriter writer = new StringWriter();

//write characters to writer.

String       data       = writer.toString();
StringBuffer dataBuffer = writer.getBuffer();
```

Please refer to the link for more information about StringWriter

http://docs.oracle.com/javase/7/docs/api/java/io/ StringWriter.html

### 1.3.3 Scenario Revisited:

John after understanding about the classes in java.io package understood that either FileOutputStream or FileWriter class can be used to store the data permanently into a File.

But is there any class in java.io package that can create a file?

Answer is Yes we have File class that supports creation of files and directories.

### 1.3.4 Using the File Class:

Java.io package includes a class known as File that provides support for creating files and dir -ectories in java

What can we do using File class?

1)Create a new file/Directory

2)Open an existing file

3)Closing a file

4)Delete a file/directory

5)Get the name,size of the file

6)Check the existence of file

7)Rename the file

8)Check whether the file is Readable and writable

9)List the contents of a directory

10)Get the absolute path of file or directory

11)Change the read/write permission of a file

12)List out all the files and folders (including all the nested files and folders) present under a Directory.

13)Print the path (absolute and relative) of a File, and the size of the File.

14)Check if a given path name represents a file or a directory.

15)Create a new folder and  a new file inside the folder, only if the specified file or folder does not exist.

16)Delete a specified file.

17)Rename a file to a new name.

18)Append data to an existing file.

**Java.io.File** represents a file on the disk,but does not actually represent the contents of the file.

File class does not have any methods for reading and writing contents into the file

Please refer the below document for complete list of methods and constructors in File Class:

### 1.3.4.1 Changing the Read/Write Permissions of the File

To set the read permission only for the owner of the file use the setReadable(boolean readable) of the File class.

setReadable(true) enables read permission only for the owner of the file.

Sample Code Snippet:

```
File f=new File("xx.txt");
 if (f.exists()) {
        boolean bval = f.setReadable(true);
         System.out.println("set the owner's read permission: "+ bval);
    } else {
         System.out.println("File cannot exists: ");
    }
}
```

To set the read permission for all the users of the file use the setReadable(boolean readable,boolean readonly )of the File class.

setReadable(true,false)enables read permission for all the users of the file.

Sample Code Snippet:

```
if (f.exists()) {
    boolean bval = f.setReadable(true,false);
    System.out.println("set the every user read permission: "+ bval);
} else {
    System.out.println("File cannot exists: ");
}
```

To set the write permission only for the owner of the file use the setWritable(boolean writable) of the File class.

setWritable(true) enables read permission only for the owner of the file.

Sample Code Snippet:

```
if (f.exists()) {
    boolean bval = f.setWritable(true);
     System.out.println("set the owner's write permission: "+ bval);
} else {
     System.out.println("File cannot exists: ");
}
```

To set the write permission for all the users of the file use the setWritable(boolean writable,boolean writeonly )of the File class.

setWritable(true,false)enables write permission for all the users of the file.

Sample Code Snippet:

```
if (f.exists()) {
    boolean bval = f.setWritable(true,false);
    System.out.println("set the every user write permission: "+ bval);
} else {
    System.out.println("File cannot exists: ");
}
```

### 1.3.4.2 List out all the files and folders present in a directory

To implement this requirement invoke the following methods:

1)Invoke isDirectory () on the file instance which returns true if it is a directory.

2)Invoke list() method on the file instance to retreive the individual files and folders present in the directory.

Sample Code Snippet:

```
File f4=new File("d:/abc");
  if(f4.isDirectory()){

        String str[] = f4.list();
        for( int i = 0; i<str.length; i++){
            File f5=new File(f4.getName() + " / " + str[i]);
            if(f5.isDirectory()){
                System.out.println(str[i] + " is a directory");
            }
            else{
                System.out.println(str[i] + " is a file");
            }
        }
    }
    else{
        System.out.println(f4.getName()  + " is not a directory");
    }
```

### 1.3.4.3 Print the path (absolute and relative) of a file, and size of the file

To retrieve the size of the file invoke length() on the file instance that returns length of the file in bytes.

Method Syntax:

Long length()

Sample Code Snippet

```
File f=new File("xx.txt");
/**
 * Retreiving  the length of the File
 */
System.out.println("The Size of the file==="+f.length());
```

To retrieve the absolute path of the file use  getAbsolutePath() method and to retreive the

relative path of the file use getPath() method.

Sample Code Snippet

```
/**
 * Retreiving the absolute and relative path of the file.
 */
File f=new File("xx.txt");
System.out.println("Absolute path of the File=="+f.getAbsolutePath());
System.out.println("Relative Path of the File===="+f.getPath());
```

### 1.3.4.4 Checking if the given path name represents file or a directory

Invocation of isFile() on the file instance returns true if it is a file and false if not file.

Invocation of isDirectory() on the file instance returns true if it is a directory and false if not directory.

Sample Code Snippet:

```
File f=new File("xx.txt");
System.out.println("Is it a file?"+f.isFile());
System.out.println("Is it a directory?"+f.isDirectory());
```

### 1.3.4.5 Create a new folder and a new file inside the folder, only if the specified file or folder does not exist.

To implement this requirement following methods must be used

1)To test whether the file or directory denoted by this abstract pathname exists make use of the below method:

boolean exists()

2)To create a directory make use of mkdir() that returns true if the directory is created and false otherwise

boolean mkdir()

3)To create directory along with all necessary parent directories use

boolean mkdirs()

Sample Code Snippet:

```
String strDirectoy="test";
String strManyDirectories="dir1/dir2/dir3";
boolean doesDirectoryExists=false;
boolean success=false;
doesDirectoryExists=new File(strDirectoy).exists();
System.out.println("Is Directory existing==="+doesDirectoryExists);
if(!doesDirectoryExists)
{
// Create one directory
 success = (new File(strDirectoy)).mkdir();
if (success) {
    System.out.println("Directory: " + strDirectoy + " created");
}
String newFile="test/newFileOne.txt";
File f1=new File(newFile);
success=f1.createNewFile();
System.out.println("File Created=="+success);
}
doesDirectoryExists=new File(strManyDirectories).exists();
```

```
// Create multiple directories
success = (new File(strManyDirectories)).mkdirs();
if (!success) {
    System.out.println("Directories: " + strManyDirectories + " created");
}
```

### 1.3.4.6 Delete a Specified File.

To implement this invoke the delete on the file instance.

Sample Code Snippet:

```
/**Deleting a specified file*/
boolean fileDeleted=f.delete();
System.out.println("fileDeleted==="+fileDeleted);
```

### 1.3.4.7 Rename a file to new file name.

boolean renameTo(File dest) can be used to rename the file denoted by the abstract path name to the new file name

Sample Code Snippet:

```
System.out.println("Renaming file to a new file name");
boolean renameDone=f2.renameTo(f3);
System.out.println("renameDone====="+renameDone);
```

### 1.3.4.8 Code snippet on File class

```
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStreamReader;


public class FileClassDemo {

	/**
	 * @param args
	 */
	public static void main(String[] args) {

		File f2=new File("D:/demo/Files.txt");
		File f3=new File("D:/demo/Files_Renamed.txt");
		File f4=new File("D:/demo");
		File f=new File("xx.txt");
		/**
		 * Retreiving  the length of the File
		 */
		System.out.println("The Size of the file==="+f.length());

		BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

		try {
			/**
			 * Retreiving the absolute and relative path of the file.
```



Sample Code Snippet demonstrating all the operations that are possible by making use of File class:

```
System.out.println("The Size of the file==="+f.length());

BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

try {
        /**
         * Retreiving the absolute and relative path of the file.
         */
        System.out.println("Absolute path of the File=="+f.getAbsolutePath()+",Relative
Path==="+f.getPath());
        /**
         * Checking if a given path name represents file or directory
         */
        System.out.println("Checking if a given path name represents file or directory");
        System.out.println("Is it a file?"+f.isFile());
        System.out.println("Is it a directory?"+f.isDirectory());
```

/**
* Renaming the file to the new filename

```
         */
        System.out.println("Renaming file to a new file name");
        boolean renameDone=f2.renameTo(f3);
        System.out.println("renameDone====="+renameDone);
        /**
         * Changing the  write permissions for the file
         */
         //set write permission on file only for owner
if (f.exists()) {
   boolean bval = f.setWritable(true);
    System.out.println("set the owner's write permission: "+ bval);
} else {
    System.out.println("File cannot exists: ");
}

//Set write permission on File for all.
if (f.exists()) {
   boolean bval = f.setWritable(true,false);
   System.out.println("set the every user write permission: "+ bval);
} else {
   System.out.println("File cannot exists: ");
}
```

```
                //set read permission on file only for owner
if (f.exists()) {
   boolean bval = f.setReadable(true);
    System.out.println("set the owner's read permission: "+ bval);
} else {
    System.out.println("File cannot exists: ");
}

//Set read permission on File for all.
if (f.exists()) {
   boolean bval = f.setReadable(true,false);
   System.out.println("set the every user read permission: "+ bval);
} else {
   System.out.println("File cannot exists: ");
}
/**
 * create a new folder and a new file inside the folder, only if the specified file
 * or folder does not exist.
 */
String strDirectoy="test";
String strManyDirectories="dir1/dir2/dir3";
boolean doesDirectoryExists=false;
boolean success=false;
```

```
doesDirectoryExists=new File(strDirectoy).exists();
  System.out.println("Is Directory existing==="+doesDirectoryExists);
  if(!doesDirectoryExists)
  {
            // Create one directory
             success = (new File(strDirectoy)).mkdir();
            if (success) {
                    System.out.println("Directory: " + strDirectoy + " created");
            }
            String newFile="test/newFileOne.txt";
            File f1=new File(newFile);
            success=f1.createNewFile();
            System.out.println("File Created=="+success);
  }
  doesDirectoryExists=new File(strManyDirectories).exists();
            // Create multiple directories
            success = (new File(strManyDirectories)).mkdirs();
            if (!success) {
                    System.out.println("Directories: " + strManyDirectories + " created");
            }
            /**
             * List Out All The Files And Folders (including All The Nested Files And
Folders) Present Under A Directory.
             */
             if(f4.isDirectory()){

                    String str[] = f4.list();
                    for( int i = 0; i<str.length; i++){
                            File f5=new File(f4.getName() + " / " + str[i]);
                            if(f.isDirectory()){
                                    System.out.println(str[i] + " is a directory");
                            }
```

```
/**Deleting a specified file*/
        boolean fileDeleted=f.delete();
System.out.println("fileDeleted===="+fileDeleted);
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }

}

}
```

### 1.3.5 Scenario Revisited:

From the above methods of File class we can understand that there are no methods that does reading or writing on the file.

How do perform read/write operations on a file?

To perform read /write operations there are two possible ways:

1)Reading /Writing in terms of Byte representation:

To achieve this we have to make use of InputStream and outputStream classes

2)Reading /Writing in terms of character representation:

To achieve this we have to make use of Reader and Writer classes

### 1.3.6 Using DataInputStream,FileOutputStream classes for the given problem scenario:

To Implement this requirement follow the below steps:

1)Attach the keyboard to the DataInputStream

```
DataInputStream dis=new DataInputStream(System.in);
```

2)Attaching File to FileOutputStream

```
FileOutputStream fout=new FileOutputStream("sample.txt",true);
```

3)Make use of DataInputStream read() method to accept the characters

4)Write the character into the file using write method of FileOutputStream

5)Close the resources once reading and writing is done.

Sample Code Snippet:

```
import java.io.DataInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

/*In this program the data is read from the keyboard and written
 *into sample.txt .The data is accepted from the keyboard until
 *the user types $ when he does not want to continue . The Point
 *to notice here is when the program is executed again the old data
 * loose the old data then we should open the file in writing mode
 * This can be acheived using the below code
 * fout=new FileOutputStream("sample.txt",true);
 *
 * Using the above data even though the program is executed many
 * times all the previous data will be retained  with new data
 *
 */
public class CreateFileDemo {
```

```
    public static void main(String[] args) {
        DataInputStream dis=null;
        FileOutputStream fout=null;
        try
        {
        //attaching keyboard to DataInputStream

         dis=new DataInputStream(System.in);

        //Attaching File to FileOutputStream to write
         fout=new FileOutputStream("sample.txt",true);

        System.out.println("Enter text to be written to the file"+
                                "enter $ to indicate the end of file");

        char ch;

        while((ch=(char)dis.read())!='$')
        fout.write(ch);
```

```
        }
        catch(IOException ioe)
        {
            ioe.printStackTrace();
        }
        finally
        {
            try {
                fout.close();
                dis.close();
            } catch (IOException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }

        }
```

Some observations in the above code snippet:

***Appending the data to Existing File:***

For appending the data to the existing file we have to open the file in Writing mode and to achieve this we have to make use of the below constructor of FileOutputStream

**FileOutputStream(String name, boolean append)**

In the above program the data is read from the keyboard and written into sample.txt .The data is accepted from the keyboard until the user types $ when he does not want to continue.

The Point to notice here is when the program is executed again the old data in the file will be lost keeping only the recent data . In case if you do not want to loose the old data then we should open the file in writing mode

This can be achieved using the below code

**fout=new FileOutputStream("sample.txt",true);**

Using the above data even though the program is executed many times all the previous data will be retained with new data

### 1.3.7 Introduction to Buffer

Buffer is temporary block of memory. Buffer gives us temporary holding place to group things until the holder is full. To increase the efficiency /performance of writing the characters we have to make use of Buffered classes.

Buffered classes must be used in connection with other stream classes.

The default buffer size used by any buffered class is 512 bytes.

### 1.3.8 Relating Buffer to Real World Scenario

Rita was asked by her mother to bring 10 items from the grocery store. Consider two scenarios here:

1) If Rita brings one item at a time from the grocery store to her house then she is unnecessarily increasing the round trip times from the grocery store to her house and also this process consumes more time.

2)If Rita collected all the items in a container to her house then she required only one trip from store to her house and also she could have saved some time

So out of the above two scenarios what do you think is an optimal one?

Second scenario right!!! Now we apply the same concept to our I/O programming as well.

If Rita uses second approach then she is actually applying the concept of buffering and hence improv-

ing the efficiency of the program.

### 1.3.9 Using DataInputStream ,BufferedOutputStream,FileOutputStream classes for the given problem scenario

To implement this requirement follow the below steps

1)Attach keyboard to DataInputStream

Syntax:

```
DataInputStream dis =new DataInputStream(System.in);
```

2)Attaching File to FileOutputStream to write

```
FileOutputStream fout=new FileOutputStream("sample.txt",true);
```

3)Attaching FileOutputStream to BufferedOutputStream

```
BufferedOutputStream bout=new BufferedOutputStream(fout,1024);
```

4)Write the buffered characters at once to file

```
while((ch=(char)dis.read())!='$')
bout.write(ch);
```

5)Close the resources once reading and writing is done.

### 1.3.10 Code Snippet using DataInputStream ,BufferedOutputStream,FileOutputStream classes

```java
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.DataInputStream;
import java.io.FileOutputStream;
import java.io.IOException;


public class CreateFileUsingBOS {
    public static void main(String[] args) {
        DataInputStream dis=null;
        FileOutputStream fout=null;
        BufferedOutputStream bout=null;
        BufferedInputStream bin=null;
        try
        {
```

```java
        //attaching keyboard to DataInputStream

         dis=new DataInputStream(System.in);

        //Attaching File to FileOutputStream to write
         fout=new FileOutputStream("sample.txt",true);
        //Attaching FileOutputStream to BufferedOutputStream
         bout=new BufferedOutputStream(fout,1024);

        System.out.println("Enter text to be written to the file "+
                "enter $ to indicate the end of file");

        char ch;

        while((ch=(char)dis.read())!='$')
```

```
        bout.write(ch);
        }
        catch(IOException ioe)
        {
            ioe.printStackTrace();
        }
        finally
        {try {
                fout.close();
                bout.close();
                dis.close();
            } catch (IOException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
}
```

In the above code snippet to improve the efficiency of writing characters into the file BufferedOutputStream class is used along with the FileOutputStream

Hence instead of writing single character into the file we write the characters into the buffer and when the buffer is full,the entire buffer of data is sent in a single step to the file.

### 1.3.11 Using BufferedInputStream ,BufferedOutputStream,FileOutputStream classes for the given problem scenario

To implement this requirement follow the below steps

1)Attach FileInputStream to BufferedInputStream

Syntax:

```
 InputStream inStream = null;
inStream = new FileInputStream("D:/demo/s/test.txt");
```

2)Attaching File to FileOutputStream to write

```
OutputStream outStream=null;
outStream= new FileOutputStream("D:/demo/s/test1.txt");
```

3)Attaching FileOutputStream to BufferedOutputStream

```
BufferedOutputStream bos=null;
bos = new BufferedOutputStream(outStream,1024);
```

4)Write the buffered characters at once to file

```
while(bis.available()>0)
{
   // read the byte and convert the integer to character
   char c = (char)bis.read();
   bos.write(c);
   bos.flush();
   // print the characters
   System.out.println("Char: "+c);
}
```

5)Close the resources once reading and writing is done.

```
finally{
   // releases any system resources associated with the
    //stream
   if(inStream!=null)
      inStream.close();
   if(bis!=null)
      bis.close();
   }
```

### 1.3.12 Code Snippet using BufferedInputStream ,BufferedOutputStream,FileOutputStream classes

```java
import java.io.*;
public class FileUsingBISBOS {
    public static void main(String[] args) throws IOException {
        InputStream inStream = null;
        BufferedInputStream bis = null;
        OutputStream outStream=null;
        BufferedOutputStream bos=null;

        try{
            // open input stream test.txt for reading purpose.
            inStream = new FileInputStream("D:/demo/s/test.txt");
            outStream= new FileOutputStream("D:/demo/s/test1.txt");
            // input stream is converted to buffered input stream
            bis = new BufferedInputStream(inStream);
            bos = new BufferedOutputStream(outStream);
            // read until a single byte is available
            while(bis.available()>0)
            {
```

```java
                // read the byte and convert the integer to character
                char c = (char)bis.read();
                bos.write(c);
                bos.flush();
                // print the characters
                System.out.println("Char: "+c);
            }
        }catch(Exception e){
            // if any I/O error occurs
            e.printStackTrace();
        }finally{
            // releases any system resources associated with the
             //stream
            if(inStream!=null)
                inStream.close();
            if(bis!=null)
                bis.close();
            }
        }
```

### 1.3.13 Introduction to BufferedReader and BufferedWriter

#### BufferedReader:

## BufferedWriter:

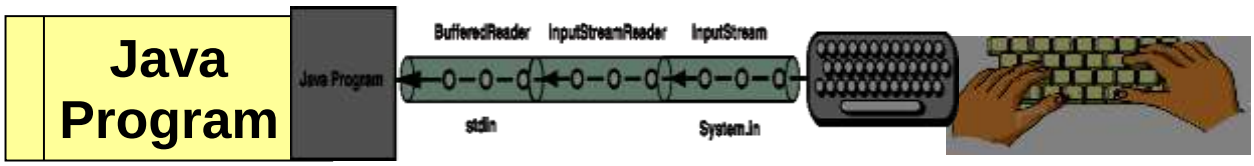
Writes text to a character-output stream, buffering characters so as to provide for the efficient writing of single characters, arrays, and strings

## 1.3.14 Performing File I/O using BufferedReader and BufferedWriter classes

To implement this requirement follow the below steps:

1)Attach keyboard to InputStreamReader

2)Attach InputStreamReader to BufferedReader




Syntax:

```
BufferedReader in = new BufferedReader( new InputStreamReader(System.in));
```

3)Create FileWriter and attach it to BufferedWriter

Syntax:

```
FileWriter fstream = new FileWriter(file_name);
BufferedWriter out = new BufferedWriter(fstream);
```

4)Read the lines using BufferedReader and write it to file using BufferedWriter.

```
out.write(in.readLine());
```

5)Close the resources once reading and writing is done.

### 1.3.15 Code snippet using BufferedReader and BufferedWriter classes

```
import java.io.*;
public class  classFileWriter{
public static void main(String[] args) throws IOException{

BufferedReader in = new BufferedReader(
        new InputStreamReader(System.in));
System.out.print("Please enter the file name to create : ");
String file_name = in.readLine();
File file = new File(file_name);
boolean exist = file.createNewFile();
if (!exist)
        {
            System.out.println("File already exists.");
            System.exit(0);
        }
else
        {
            FileWriter fstream = new FileWriter(file_name);
            BufferedWriter out = new BufferedWriter(fstream);
```

```
            System.out.println("Please enter the characters to be"
                    +" written to the File..");
            out.write(in.readLine());
            out.close();
            System.out.println("File created successfully.");
        }
    }
}
```

## 1 1.4 Conclusion-- Choosing the right I/O implementation class.

The below table depicts the scenarios and the respective I/O class for handling these scenarioes. Table also covers other implementation classes which are not discussed in the course.

| Sno | Scenario | Byte Stream class | Character Stream Class |
|---|---|---|---|
| 1 | Buffering Input | BufferedInputStream | BufferedReader |
| 2 | Keeping track of line numbers | LineNumberInputStream | LineNumberReader |
| 3 | Reading from Array | ByteArrayInputStream | CharArrayReader |
| 4 | Reading from Files | FileInputStream | FileReader |
| 5 | Filtering the Input | FilterInputStream | FilterReader |
| 6 | Pushing back characters or bytes | PushbackInputStream | PushbackReader |
| 7 | Reading from a pipe | PipedInputStream | PipedReader |
| 8 | Reading from a String | StringBufferInputStream | StringReader |
| 9 | Reading Primitive types | DataInputStream | none |
| 10 | Translating Byte Stream into a character stream | InputStreamReader | none |
| 11 | Buffering output | BufferedOutputStream | BufferedWrite r |
| 12 | Writing to an array | ByteArrayOutputStream | CharArrayWriter |
| 13 | Writing to a file | FileOutputStream | FileWriter |
| 14 | Filtering the output | FilterOutputStream | FilterWriter |
| 15 | Printing values and objects | PrintStream | PrintWriter |
| 16 | writing to a pipe | PipedOutputStream | PipedWriter |
| 17 | Writing to a String | | StringWriter |
| 18 | Writing primiive | DataOutputStream | none |

| | | | |
|---|---|---|---|
| | Types | | |
| 19 | Translating Character Stream to a byte Stream | none | OutputStreamWriter |

## 1.5 Check your self

1._____adds functionality to another input stream-namely, the ability to buffer the input

A.BufferedInputStream

B.InputStream

C.BufferedWriter

D.DataInputStream

2._____exception is thrown by read() method of InputStream class

A.IOException

B.FileNotFoundException

C.ReadException

D.None of the above

3._____lets an application write primitive Java data types to an output stream in a portable

way.

A. DataOutputStream

B. DataInputStream

C.BufferedInputStream

D.FileInputStream

4.Can a FileOutputStream instance be attached to BufferedOutputStream?

A.True B.False

5._____method is used to check the length of the file

A.size

B.length

C.total

D.None of the above

6.setReadable(boolean) method when passed with true value sets the read permission for the file?

A.true  B.false

7._____ method is used to list the files and directories in a file

A.list

b.total

c.listAll

d.display

8._____signals that an I/O exception of some sort has occurred.

A. IOException

B.FileNotFoundException

C.ReadException

D.None of the above

9.java.io.File  is an abstract representation of file and directory path names.

A.True  B.False

10.An InputStreamReader is a bridge from byte streams to character streams

A.True  B.False

## 1.6 Best Practises

1)Use java.io.File instead of String for file names to avoid platform dependencies

Let's look at an example.

```
String getExtension(String filename) {
        int index = filename.lastIndexOf('.');
        if (index == -1) {
          return "";
        } else {
          return filename.substring(index + 1);
        }
      }
```

Easy enough? Right, but what happens if someone passes in a full path instead of only a filename? Consider the following, perfectly legal path: "C:\Temp\documentation.new\README". The method as defined above would return "new\README" - definitely not what you wanted.

Please use java.io.File for filenames instead of Strings. The functionality that the class provides is well tested. In FileUtils you will find other useful utility functions around java.io.File.

Instead of:

```
String tmpdir = "/var/tmp";
 String tmpfile = tmpdir + System.getProperty("file.separator") + "test.tmp";
 InputStream in = new java.io.FileInputStream(tmpfile);
```

```
File tmpdir = new File("/var/tmp");
File tmpfile = new File(tmpdir, "test.tmp");
InputStream in = new java.io.FileInputStream(tmpfile);
```

2)Buffering Streams

IO performance depends a lot from the buffering strategy. Usually, it's quite fast to read packets with

the size of 512 or 1024 bytes because these sizes match well with the packet sizes used on harddisks in file systems or file system caches. But as soon as you have to read only a few bytes and that many times performance drops significantly.

Make sure you're properly buffering streams when reading or writing streams, especially when working with files. Just decorate your FileInputStream with a BufferedInputStream:

```
InputStream in = new java.io.FileInputStream(myfile);
 try {
   in = new java.io.BufferedInputStream(in);

   in.read(.....
 } finally {
   IOUtils.closeQuietly(in);
 }
```

## 1.7 Take Away Points

1)A BufferedInputStream adds functionality to another input stream-namely, the ability to buffer the input

2)BufferedReader reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines

3)BufferedWriter writes text to a character-output stream, buffering characters so as to provide for the efficient writing of single characters, arrays, and strings.

4)DataInputStream lets an application read primitive Java data types from an underlying input stream in a machine-independent way.

5)DataOutputStream lets an application write primitive Java data types to an output stream in a portable way.

6)java.io.File is an abstract representation of file and directory pathnames.

7)FileReader is convenience class for reading character files.

8)FileWriter is convenience class for writing character files.

9)InputStream is an abstract class and it is the superclass of all classes representing an input stream of bytes.

10)OutputStream is abstract class and it is the superclass of all classes representing an output stream of

bytes.

11)An InputStreamReader is a bridge from byte streams to character streams: It reads bytes and decodes them into characters using a specifiedcharset.

12)Reader is an abstract class for reading character streams.

13)Writer is an abstract class for writing to character streams.

14)IOException signals that an I/O exception of some sort has occurred.

## 1.8 Related Topics

Serialization

## 1.9 References

http://commons.apache.org/proper/commons-io/bestpractices.html
http://tutorials.jenkov.com/